Experiment 1: Linux Command Execution Using fork() and exec()

Course: Operating Systems and Systems Programming (OSSP)

Name: Akshita Anand

Roll No.: 2520030043

Date: 7/30/2026

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Experiment Steps

Step 1: Check Whether GCC is Installed

To check if the GNU C Compiler (GCC) was installed, I executed the following command:

gcc --version

```
akshi@akshita:~$ gcc --version
Command 'gcc' not found, but can be installed with:
sudo apt install gcc
```

Observation

The terminal indicated that the GCC compiler was not installed on the system.

Step 2: Install GCC

I installed the GCC compiler using the following commands:

sudo apt install build-essential

```
akshi@akshita:~$ sudo apt update
[sudo: authenticate] Password:
Get:1 http://archive.ubuntu.com/ubuntu resolute InRelease [136 kB]
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [136 kB]
Get:5 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [341
Get:6 http://archive.ubuntu.com/ubuntu resolute/main amd64 Packages [1480 kB]
Get:7 http://archive.ubuntu.com/ubuntu resolute/main Translation-en [524 kB]
Get:8 http://archive.ubuntu.com/ubuntu resolute/main amd64 Components [395 kB]
Get:9 http://archive.ubuntu.com/ubuntu resolute/main amd64 c-n-f Metadata [32.4 kB
Get:10 http://archive.ubuntu.com/ubuntu resolute/universe amd64 Packages [16.0 MB]
Get:11 http://archive.ubuntu.com/ubuntu resolute/universe Translation-en [6329 kB]
Ign:12 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en
```

Step 4: Create the Source File

Created a C source file named **shell_sim.c** using the Nano editor.

The Nano text editor opened, and the required C program was typed into the file.

```
akshi@akshita:~$ nano shell_sim.c
```

Step 5: Save and Exit the File

After entering the C program, the file was saved and the Nano editor was closed using the following keyboard shortcuts:

- Pressed **Ctrl + O** to save the file.
- Pressed **Enter** to confirm the filename (shell_sim.c).
- Pressed **Ctrl + X** to exit the Nano editor.

Step 6: Verify the Source Code

To verify that the program was saved successfully, the following command was executed:

cat shell_sim.c

The terminal displayed the complete C program stored in the shell_sim.c file, confirming that the source code was saved correctly.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    char command[100];

    printf("Enter Linux Command: ");
    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    pid_t pid = fork();

    if (pid < 0)
    {
        printf("Fork failed!\n");
        return 1;
    }
    else if (pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID = %d\n", getpid());

        execlp(command, command, NULL);

        printf("Command not found!\n");
    }
    else
    {
        printf("\nParent Process\n");
        printf("Parent PID = %d\n", getpid());

        wait(NULL);

        printf("Child execution completed.\n");
```

Step 7: Compile the Program

The source code was compiled using the GCC compiler

```
~$ sudo apt install build-essential -y
```

Step 8: Execute the Program

The compiled program was executed using:

./shell_sim

When prompted:

Enter Linux Command:

I entered:

1) **Ls**

```
akshi@akshita:~$ gcc -o shell_sim shell_sim.c
akshi@akshita:~$ ./shell_sim
Enter Linux Command: ls

Parent Process
Parent PID = 2425

Child Process
Child PID = 2426
OSSP  shell_sim  shell_sim.c
Child execution completed.
```

2) **pwd**

```
Enter Linux Command: pwd

Parent Process
Parent PID = 2436

Child Process
Child PID = 2437
/home/akshi
Child execution completed.
```

3) **date**

```
Enter Linux Command: date

Parent Process
Parent PID = 2447

Child Process
Child PID = 2450
Thu Jul 30 05:22:58 UTC 2026
Child execution completed.
```

4) **Uname**

```
akshi@akshita:~$ uname -a
```

5)lscpu

```
akshi@akshita:~$ lscpu
Architecture:                x86_64
  CPU op-mode(s):            32-bit, 64-bit
  Address sizes:             42 bits physical, 48 bits virtual
  Byte Order:                Little Endian
CPU(s):                      8
  On-line CPU(s) list:       0-7
Vendor ID:                   GenuineIntel
  Model name:                Intel(R) Core(TM) Ultra 7 256V
    CPU family:              6
    Model:                   189
    Thread(s) per core:      1
    Core(s) per socket:      8
    Socket(s):               1
    Stepping:                1
    BogoMIPS:                6604.80
    Flags:                   fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall n
                             x pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pclmul
                             qdq vmx ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hyperv
                             isor lahf_lm abm 3dnowprefetch ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjust
                             bmi1 avx2 smep bmi2 erms invpcid rdseed adx smap clflushopt clwb sha_ni xsaveopt xsavec xgetbv1 xsaves avx_vnni v
                             nmi umip waitpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b fsrm md_clear serialize ibt flush_l1d arch_capabili
                             ties
Virtualization features:
  Virtualization:            VT-x
  Hypervisor vendor:         Microsoft
  Virtualization type:       full
Caches (sum of all):
  L1d:                       384 KiB (8 instances)
  L1i:                       512 KiB (8 instances)
  L2:                        20 MiB (8 instances)
  L3:                        12 MiB (1 instance)
NUMA:
  NUMA node(s):              1
  NUMA node0 CPU(s):         0-7
Vulnerabilities:
  Gather data sampling:      Not affected
  Ghostwrite:                Not affected
  Indirect target selection: Not affected
  Itlb multihit:             Not affected
  L1tf:                      Not affected
  Mds:                       Not affected
  Meltdown:                  Not affected
  Mmio stale data:           Not affected
  Old microcode:             Not affected
  Reg file data sampling:    Not affected
  Retbleed:                  Mitigation; Enhanced IBRS
  Spec rstack overflow:      Not affected
  Spec store bypass:         Mitigation; Speculative Store Bypass disabled via prctl
  Spec v1:                   Mitigation; usercopy/swapgs barriers and __user pointer sanitization
  Spec v2:                   Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSB-eIBRS SW sequence; BHI BHI_DIS_S
  Srbds:                     Not affected
  Tsa:                       Not affected
  Tsx async abort:           Not affected
  Vmscape:                   Not affected
```

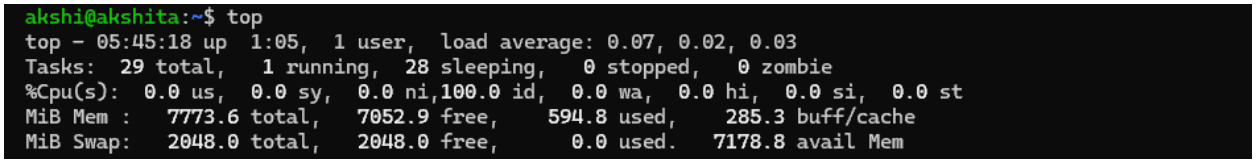
6)lsblk

```
akshi@akshita:~$ lsblk
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda    8:0    0 356.9M  1 disk
sdb    8:16   0 159.4M  1 disk
sdc    8:32   0     2G  0 disk [SWAP]
sdd    8:48   0     1T  0 disk /mnt/wslg/distro
```

7)Ps

```
akshi@akshita:~$ ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.1  23820 15004 ?        Ss   04:39   0:01 /usr/lib/systemd/sy
root           2  0.0  0.0   3180  2204 hvc0     Sl+  04:39   0:00 /init
root           6  0.0  0.0   3180  2044 hvc0     Sl+  04:39   0:00 plan9 --control-soc
root          46  0.0  0.2  50392 16820 ?        S<s  04:39   0:00 /usr/lib/systemd/sy
systemd+      78  0.0  0.1  22540 14592 ?        Ss   04:39   0:00 /usr/lib/systemd/sy
root          90  0.0  0.1  35320 12228 ?        Ss   04:39   0:01 /usr/lib/systemd/sy
root         139  0.0  0.0   2888  1948 ?        Ss   04:39   0:00 /bin/sh /usr/lib/sy
root         140  0.0  0.0   4472  3168 ?        Ss   04:39   0:00 /usr/sbin/cron -f -
```

8)top


akshi@akshita:~$ top
top - 05:45:18 up 1:05, 1 user, load average: 0.07, 0.02, 0.03
Tasks: 29 total,   1 running, 28 sleeping,   0 stopped,   0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem :   7773.6 total,   7052.9 free,   594.8 used,   285.3 buff/cache
MiB Swap: 2048.0 total,   2048.0 free,   0.0 used.   7178.8 avail Mem